Observability Systems: Answers to Three Questions

Logs, Metrics, Traces · Log Storage · Pull and Push Metrics Collection

Answering approach Each section begins with a direct comparison, then explains scaling behavior and engineering causes, and ends with a concrete architectural choice and its limits.

Answer map

1

Logs, Metrics, and Traces

Record structure, storage growth, best-fit questions, and why a telemetry pipeline is a high-throughput distributed system.

2

Two Families of Log Storage

Write and read costs, fast and slow query patterns, high-cardinality failure modes, and the right choice for high-volume recent-service searches.

3

Pull and Push Metrics Collection

Service discovery, down detection, short-lived jobs, network boundaries, rate control, and the impact of a misbehaving application.

One-sentence overview

Logs preserve the context of discrete events, metrics compress system behavior into numerical trends, and traces reconstruct the causal path of individual requests. They complement one another rather than compete.

Question 1: Data Models of Logs, Metrics, and Traces

Core conclusion Log volume is driven mainly by event count; metric volume by the number of active time series, meaning distinct label combinations; and trace volume by sampled requests multiplied by the spans produced per request.

Signal	One record contains	How storage grows	Best question	Main cost
Logs	One event: timestamp, severity, message, service or instance, structured fields, and optionally trace_id and span_id.	Approximately event count x average record size. Events usually grow with traffic and log statements per request. Full-text indexing adds index amplification.	What exactly happened? What was the error context, exception stack, parameter, or discrete business event?	Rich detail but high cost; less efficient than metrics for repeated aggregation.
Metrics	One time-series sample: metric name, complete label set, timestamp, and value. Histograms create bucket, count, and sum series.	Active label combinations x sample rate x retention. Cardinality is the key driver; more traffic alone does not necessarily add samples.	What are the rate, ratio, percentile trend, capacity level, and SLO status over time? Best for dashboards and alerts.	Loses per-event context; high-cardinality labels rapidly multiply series.
Traces	The basic record is a span: trace, span, and parent IDs; service and operation; start and duration; status; attributes; and events. A trace is a group of spans.	Sampled requests x spans per request x average span size. It grows with traffic and with the number of services, databases, queues, and other hops touched.	Where did this request go? Which hop consumed the time? Which dependency caused the failure?	Full retention is expensive, so sampling is common; context propagation must work across services.

Scaling formulas and the key distinction

Log bytes ~ request rate x log events per request x average log size x retention

Metric samples ~ active time series x samples per second x retention

Active time series ~ metric names x distinct combinations of label values

Trace bytes ~ request rate x sampling rate x spans per request x average span size x retention

Traffic and label combinations play different roles. If request volume doubles, log events and unsampled spans normally grow by about the same factor. A fixed CPU metric, however, may still emit only one sample every 15 seconds. Conversely, adding user_id, request_id, or full URLs as metric labels can explode active series even when traffic is unchanged.

How to interpret a record

• Log: event-centered. It records a discrete fact with the context needed for investigation. Structured fields support filtering; original text remains useful to people.

• Metric: time-series-centered. The metric name plus the complete label set defines a series; each new record appends only a timestamp and value.

• Trace: causality-centered. Each span represents one operation, and parent_id links spans into a call tree. More fan-out and more services therefore produce more spans per request.

Which signal fits which question?

Question	Primary signal	Why
Did the error rate exceed 1% in the last five minutes?	Metrics	Rate and aggregation queries over fixed dimensions are cheap and suitable for continuous alerting.
Why did order 8347 fail, and what was the exception?	Logs	The investigation needs exact event context and the original error content.
Was this request slow in the API, database, or payment service?	Traces	The answer requires a cross-service timeline, parent-child relationships, and the critical path.
When did an incident start, how broad is the impact, and what sample shows the cause?	Metrics -> traces/logs	Use metrics to find scope, then follow an exemplar or trace_id to detailed evidence.

Why the telemetry pipeline is a high-throughput, write-heavy, low-read-frequency distributed system

Every request, background job, host, and network device continuously emits telemetry. Writes continue even when nobody is looking at a dashboard. Human queries, alert evaluations, and reports are comparatively sparse. The baseline load is therefore a large stream of append-oriented writes plus fewer reads that may scan broad time ranges.

Property	Engineering reason
High throughput	Many instances and regions write concurrently. Peaks often coincide with incidents because retries and extra error logging amplify the stream.
Write-heavy	Each record is received, parsed, batched, compressed, partitioned, replicated, and indexed or placed in object storage. Many records are written once and never read.
Low read frequency	Most retained history is never queried by a person. Reads come from dashboards, alerts, investigations, or offline analysis and occur far less often than writes.
Distributed by necessity	One machine cannot carry total throughput, capacity, and failure risk. The system needs sharding, replication, tenant isolation, horizontal scaling, and multi-zone durability.
Flow control	Queues, batching, backpressure, retries, rate limits, sampling, and drop policies prevent a slow backend from pushing failure back into production services.

Typical path: application or agent -> collector -> queue or stream -> processing and indexing -> hot store or object storage

Design goals: horizontal throughput, efficient batches, balanced partitions, retryable failures, tenant quotas, and controlled degradation.

Compressed answer Logs answer event questions, metrics answer trend questions, and traces answer path questions. Their scale is dominated by event count, label combinations, and sampled requests x spans per request, respectively. A telemetry pipeline is distributed because it must absorb continuous multi-source writes while providing fault tolerance and scalable storage.

Question 2: Two Families of Log Storage

Core conclusion A full-text inverted index performs more work at write time to make flexible content search fast. A label index with compressed object chunks indexes only a few labels at write time, lowering ingestion cost but potentially paying for object reads, decompression, and content scans at query time.

Dimension	Full-text inverted index	Label index + compressed chunks + object storage
Write-time cost	Parse fields and text, tokenize content, maintain term-to-posting mappings, build doc values, write segments, and merge them in the background. CPU, memory, random I/O, and storage amplification are substantial.	Index only selected labels and time ranges. Logs sharing one label set form a stream; their bodies are compressed into time-ordered chunks and written to low-cost object storage. Ingestion is lighter and compression is better.
Read-time cost	Look up the term dictionary and posting lists to locate matching documents. Cost is then dominated by Boolean combination, sorting, aggregation, and fetching hits.	Use labels and time to choose candidate streams and chunks, then download, decompress, and scan lines with substring, regex, parser, or field filters. Weak selectors cause broad scans.
Fast queries	Keywords, phrases, indexed-field filters, and Boolean combinations. Interactive exploration of unknown fields is generally strong.	A selective label plus a narrow time window, such as service=checkout for the last 15 minutes. A body filter remains fast when only a few chunks survive pruning.
Slow queries	Leading wildcards, complex regex, script queries, high-cardinality aggregations over a broad range, cross-shard fan-out, and deep pagination.	No useful label, a long time range, full-text or complex regex searches across many streams. These require scanning many compressed chunks.
High-cardinality damage	Unique terms or values enlarge the term dictionary and posting metadata, increase segment, merge, heap, and cache pressure, and can increase shard fan-out.	Making request_id or user_id a label creates huge numbers of streams, tiny chunks, poor compression, a larger label index, and more object requests.
Typical system	Elasticsearch or OpenSearch style log search built around full-text indexing.	Loki style design: low-cardinality labels locate streams while the log body remains in compressed object chunks.

The two designs pay at different stages

• Full-text inverted index: write-heavy indexing. The system organizes future search terms and fields during ingestion, so storage amplification, merges, and memory cost are high; queries can then jump directly to a small set of documents.

• Label index with object chunks: scan on read. Ingestion builds only a small index and compresses the body efficiently; if labels cannot narrow the candidate set, the cost moves to object reads, decompression, and scans.

High cardinality hurts both families

The failure mechanisms differ. Inverted indexes accumulate a larger term dictionary, more posting metadata, more segments, and a bigger cache working set. Label-based systems split logs into enormous numbers of streams, so chunks are flushed before they grow, producing many small objects, poor compression, and more index entries.

Safe labels: service, environment, region, cluster, level - small and stable value sets

Dangerous labels: request_id, trace_id, user_id, session_id, full URL - values are close to unique per event

Keep dangerous values in the log body or structured fields, but do not make them stream labels.

Scenario choice: very high volume, mostly 'this service, last 15 minutes'

> Choose the label index + compressed chunks + object storage family Use low-cardinality labels such as service, environment, and region. The service selector is highly selective and the 15-minute window limits chunk count, so queries read only a small recent set. Avoiding a full-text index for every log line also gives much better ingestion economics at very high volume.

The choice follows from four points:

• Query fit: the service label prunes streams first, and the narrow time window removes most chunks.

• Write fit: at extreme volume, not indexing every term avoids much of the CPU, merge, and storage amplification of full-text indexing.

• Storage fit: compressed chunks in object storage scale simply and cheaply, making longer retention practical.

• Governable risk: fix the label set, prohibit request_id and user_id as labels, and always narrow by service and time before filtering body content.

This does not mean the inverted index is always inferior. If users need interactive searches over arbitrary words and fields across months of data, or if forensic requirements demand flexible full-text discovery, an inverted index may be appropriate. A tiered design can also keep a small hot subset in a full-text index while retaining the historical body in object storage.

Scoring summary

Family	Write cost	Read cost	Best access pattern
Full-text inverted index	High: tokenization, postings, segments, merges, and index amplification	Low to medium for indexed predicates; complex wildcards and aggregations are still expensive	Free-form search and exploratory combinations over unknown content
Label index + chunks	Low: small label index, sequential batches, compression, and object storage	Depends on pruning: low with strong labels, high when selectors are weak	Known service or environment plus a narrow recent time window

Question 3: Pull-Based and Push-Based Metrics Collection

Core conclusion Pull lets a collector visit targets on a schedule, providing strong central rate control and direct evidence that a target is down. Push lets an application or agent initiate delivery, which is convenient for short-lived jobs and difficult network boundaries, but silence and traffic control are harder. Large platforms commonly combine both.

Dimension	Pull-based	Push-based
Service discovery	The collector must know target identities and addresses, usually through Kubernetes, Consul, cloud APIs, or static configuration. Discovery directly creates scrape targets.	Senders need only the collector or agent address, so the center need not discover every application instance. The sender still needs configuration or discovery for the receiving endpoint.
Detecting a down target	A strength. The collector actively scrapes and records connection failure, timeout, and up=0. Missing samples also have staleness semantics.	Weaker. Silence can mean a crash, no events, a network break, SDK failure, or buffering. Heartbeats, last-seen metrics, or an independent health check are required.
Short-lived batch jobs	A weakness. A job may finish before the next scrape. Options include shorter intervals, a local sidecar, or sending final status through an intermediate gateway.	A strength. The job can send final metrics before exit, but it still needs flush, acknowledgement, retry, and idempotency behavior to prevent loss at termination.
Firewalls and topology	The collector must reach every target inbound. Routing and security rules are complex across VPCs, NAT, edge devices, or customer networks.	Applications usually make outbound connections, which cross NAT and strict inbound firewalls more easily. Local agents or regional gateways provide fan-in.
Who controls rate	The collector centrally controls scrape interval, timeout, concurrency, and retries. It can lower frequency consistently and plan capacity centrally.	The sender usually controls emission, batching, and send frequency. The collector can constrain it only with quotas, backpressure, 429 responses, rate limits, or drops.
Misbehaving application	The application cannot arbitrarily increase scrape frequency. Timeouts, body-size limits, and concurrency isolation bound one target, although a slow or huge endpoint still consumes workers and bandwidth.	A client can retry without limit, open many connections, send too fast, or create high cardinality, directly flooding the collector. Authentication, tenant quotas, bounded queues, rate limits, and drop policy are essential.

How pull behaves

The collector discovers a target list, then scrapes each target on a centrally defined schedule. A scrape success or failure is itself a monitoring signal, which makes reachability clear. The collector also owns the pace, can time out slow targets, limit large responses, and shard target sets across collectors.

The central limitation is network direction. A collector must reach the workload. Across network boundaries, a common design places a local collector in each region and remote-writes the result to a central service. Pure periodic scraping can entirely miss a very short job.

How push behaves

An application SDK or local agent sends metrics to a collector. This fits short jobs, mobile or edge environments, and networks that permit only outbound connections. However, silence has no single meaning, so the platform must separately monitor sender heartbeat, queue age, last successful send time, and dropped-sample counts.

Push gives part of the pacing decision to the sender. A faulty client can generate both excess throughput and excess cardinality. The receiver therefore needs per-tenant authentication, sample-rate limits, maximum label counts and value lengths, bounded queues, and overload shedding.

A common hybrid architecture

Workload	Recommended pattern	Reason
Long-running services	Local Prometheus or agent pulls	Central rate control and direct observation of scrape success.
Short batch job	Job pushes to a node agent or Pushgateway	Prevents the job from disappearing before a scrape; lifecycle and stale-metric cleanup must be explicit.
Regional aggregation	Regional collector accepts pull and push, then remote-writes centrally	Keeps complicated routing local and avoids central inbound reachability to every workload.
Central platform	Receives pushed, aggregated samples	Enforces tenant authentication, quotas, backpressure, deduplication, and durable queues.

Selection rule Prefer pull for stable, long-running, reachable services when central pacing and unambiguous down detection matter. Prefer push to a local agent or gateway for short jobs, outbound-only networks, or edge deployments. At scale, combine the two instead of treating the choice as absolute.

Final quick-reference table

Question	One-paragraph answer
Question 1	Logs = events; metrics = time series; traces = request causality. The main growth drivers are event volume, label combinations, and sampled requests x spans.
Question 2	Full-text indexing pays more on writes for fast flexible reads. Label-indexed chunks write cheaply and scan at read time. Choose the label family for high-volume, service + last-15-minutes searches.
Question 3	Pull provides direct down detection and central pacing. Push handles short jobs and difficult networks but needs heartbeats, quotas, backpressure, and overload protection.

For a shorter exam response, use each section's core conclusion plus its comparison table. For a full-credit essay, add the scaling formulas, failure mechanisms, and scenario-specific choice.